

A taxonomy of Ruby calls

Alan Wu

Alan Wu



- Generalist bug fixer. Received commit privileges in 2019 🙏
- Worked on YJIT since conception at Shopify



Why categorize calls?

- It's fun! We know lots of words, but it's hard to list all of them.
- A lens for looking at Ruby's evolution:
 - 2.7: argument forwarding (`...`) and `ruby2_keywords`
 - 3.0: fully separated keywords
 - 3.1: anonymous block parameter (`&`)
 - 3.2: anonymous rest and keyword rest parameter (`*` and `**`)
 - 3.3: rule change around anonymous rest parameter forwarding in blocks
 - 3.4: argument forwarding optimization using dynamic stack layout
- Useful for fuzzing (randomly generating test cases) for testing JIT compilers



The title and this slide are the only place I'll mention the word "taxonomy", since I don't want to pretend this is more serious than it is. This talk is just about grouping different kinds of calls into categories. I like the animal analogy because the calls are like abstract animals with unique behaviour. So, welcome to the call zoo; I hope you have fun.

Since calls are everywhere in ruby code, language changes often end up touching something related to calls. There's been at least one call related change per year in the last 6 years of ruby releases. These categories give you a subsection of ruby's semantics. They might help you understand language changes or show you corners of the language you didn't know about. For practical development purposes, both the process of making categories and the categorization we make at the end help with finding bugs in the ruby implementations.

Categorization based on work required for arguments passing

- Out of scope: method lookup semantics
- Consider a particular call site: `foo(1, 2)`
- The work to satisfy the following are different
 - `def foo(*)`
 - `def foo(x, y)`
- To know the minimum work required, the body of the method is also relevant
 - But let's only consider what's in the parameter list -- it's still relevant for advanced analyses
- My bias: strong CRuby bend

So my categorization is based on the nature of the work required to pass arguments in the call. I'll be assuming that we know both the caller and the callee. There's some subtleties in finding where a call should land, but that's for a different talk.

So, argument passing. Using different features at a call site will do different kind of work, but who you're calling also matters a lot. In this `foo` example, the version of foo that takes a rest parameter requires an allocation of array, whereas the bottom one does not.

Now, if we analyze the body of the method and see that it doesn't use a parameter, we could say that no work is required for passing that argument. Also, depending on the usage, you can work backwards and optimize how the caller passes arguments. But these are somewhat second order effects, so for most of this talk I'm putting just the parameter list. It's incomplete ruby code, but it helps us focus.

A lot of this is CRuby specific, so when I say something happens, just note that the whole talk is a mix of implementation concerns and language semantics discussions.

Build your own caller-callee pair!

Call site features:

x	plain
*args	splat
x: 0	keyword
**kwargs	keyword splat
&x	block
...	forwarding
super	
yield	



Parameter features:

x	required ("lead")
x = expression	optional
*rest	rest
*	anonymous rest
x, *, post	post
x:	keyword
**kws	keyword rest
**	anonymous keyword rest
**nil	reject keywords
&block	block
...	forwardable
ruby2_keywords	ruby2_keywords
x, y	autosplat
... and more!	

To make a call pair, you'd pick features you want on the caller side and on the callee side like in a sandwich shop. You can pick multiple features for each side, so we have a combinatoric explosion. I didn't do the counting exercise to find the exact number of combinations possible, but it's not quite the power set of these since some features are mutually exclusive. <2s>

Making this table was a good exercise for me, and it hints at some obscure features like how `super` passes block arguments differently from normal calls, but then I realized that just listing off combinations of this table wouldn't be very engaging. Luckily this isn't the full story.

Syntactic form not the full story

- `foo(*args)` could entail many things depending on `args`
 - when `!args.is_a?(Array)`
 - conversion by calling `args.to_ary`
 - when `args.last.is_a?(Hash) && Hash.ruby2_keywords_hash?(args.last)`
 - depending on the callee: use as keyword arguments OR duplicate to untag the hash
 - when `nil`
 - discard the argument

Many features work differently depending on the actual arguments you pass. I picked splat -- star args here because it does a lot. When ``args`` is not an array, there is a whole conversion protocol that involves calling a method and checking the result. Also, what's inside of ``args`` also matters since we always need to check the last element in case it's a `ruby2_keywords` hash.

Specializing argument passing for a call pair

```
def foo(*)
  foo(*args)

  else if (ci_flag & VM_CALL_ARGS_SPLAT) {
    // f(*a)
    args->rest_index = 0;
    args->rest = locals[--args->argc];
    int len = RARRAY_LENINT(args->rest);
    given_argc += len - 1;

    if (!kw_flag && len > 0) {
      rest_last = RARRAY_AREF(args->rest, len - 1);
      // Check for ruby2_keywords hash in the splat
      if (RB_TYPE_P(rest_last, T_HASH) && FL_TEST_RAW(rest_last, RHASH_PASS_AS_KEYWORDS)) {
        // def f(**kw); a = [..., kw]; g(*a)
        splat_flagged_keyword_hash = rest_last;
        if (!(!RHASH_EMPTY_P(rest_last) || ISEQ_BODY(iseq)->param.flags.has_kw) || (ISEQ_BODY(iseq)->param.flags.has_kwrest)) {
          // ...
        }
      }
    }
  }
```

static for the call pair

likely stable for the call site

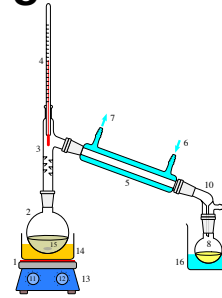
partial evaluation, 二村射影

We care about what exactly happens with these data-variant features because this is a way to find optimization opportunities. For example, you can specialize argument passing for each known call pair. This is some excerpt of argument passing code having to do with splat. Don't worry about parsing this code; the point is just there is a lot of checks in there are checking the caller side and callee side features of the pair. If we specialize based on the call pair, all these checks can go away.

Specializing argument passing for a call pair

```
def foo(*)
  foo(*args)
else if (      true      ) {
  // f(*a)
  args->rest_index = 0;
  args->rest = locals[--args->argc];
  int len = RARRAY_LENINT(args->rest);
  given_argc += len - 1;

  if (true && len > 0) {
    rest_last = RARRAY_AREF(args->rest, len - 1);
    // Check for ruby2_keywords hash in the splat
    if (RB_TYPE_P(rest_last, T_HASH) && FL_TEST_RAW(rest_last, RHASH_PASS_AS_KEYWORDS)) {
      // def f(**kw); a = [..., kw]; g(*a)
      splat_flagged_keyword_hash = rest_last;
      if (!(RHASH_EMPTY_P(rest_last) || false) || false) {
        // ...
      }
    }
  }
}
```



partial evaluation, 二村射影

The code gets reduced into something like this. Calls are fairly static in ruby programs, and this transformation removes the static parts, leaving only runtime checks on inputs. In this case it's checking the contents of the splat array.

This reduction sort of leaves behind the essential work for argument passing for the call pair. `ruby2_keywords` checks are left over, and that shows every splat call has to check the end of the array, just in case there is a keywords hash. Ideally when you don't use a feature, you don't pay a speed penalty for it, but all splats have to do this check even if you don't use `ruby2_keywords` anywhere in your code.

Anyways, you need a pretty complex JIT compiler to do this transformation mechanically, and this is not a JIT talk. This is just a way to tease semantics out of the implementation, and I'm using it to come up with my categorization.

By the way, there is a mechanism in the interpreter called fast path, where it caches a function pointer that does the argument passing logic specialized for a call pair. The body of each fast path function is more or less the output of this kind of reduction.

In-place argument passing

```
def foo(x, y, z)
```

```
foo(1, 2, 3)
```

- Method parameters are local variables
- Callers compute arguments in a layout that the callee expects. No copying!
- Only for simple call pairs (common!)

3
2
1

Right before call

<flags>
<no block>
<method : foo>
3
2
1

z

y

x

At entry to foo

The rest of the talk are just categories I made up. The first one is sort of the base-case, since it does no work for argument passing per-se. Sasada-san designed the VM so that callers compute arguments and put them on the postfix notation stack in the exact layout required parameters want. This happens before the actual call, on the caller side. When the call happens, the arguments are already in the right place, there is no need to copy or move them anywhere.

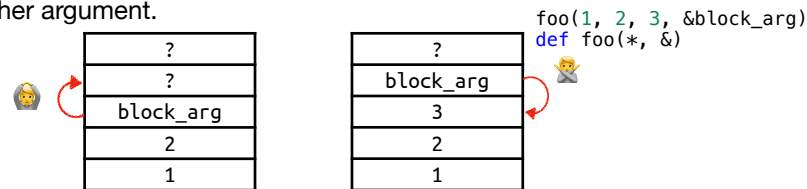
You can see that there is some metadata on top of the arguments after the call that is common to all calls, but 1,2,3 don't move. The metadata turn temporary values on the caller side into local variables in the callee side without moving them.

I'm using simple integers for this example, but this works for all expressions. This is a somewhat fundamental protocol the whole VM is designed around. These are the simplest kind of parameters you can have, so it makes sense to have them nice and efficient.

Calls that shrink the stack

```
foo(*empty_array)  foo(*[empty_ruby2_keywords_hash])  foo(*nil)
foo(**nil)  foo(**empty_hash)  def callee(*rest)  def callee(**kwrest)
```

- Stack size relevant for stack overflow check
- For callers that pass a block argument e.g. `foo(1, 2, &block_arg)`, the block argument is at the top of the stack
- YJIT wants to pop the block argument before processing other arguments and parameter features. If the stack shrinks, the final destination can collide with another argument.



Sometimes we have to move stuff, though. That means the postfix notation stack can be smaller after the call than before. There are quite a few features that can do this and I listed some examples up there. Basically, anything that requires discarding arguments like unpacking an empty array or gathering multiple arguments into one object can do this. We always want to know the stack size after the call to check for stack overflow, but also, there're cases where YJIT needs special handling for shrinking. Computation in the VM always use values at the top of the postfix notation stack as input; this is how VM operations are defined. YJIT follows this while handling calls, so once it's done with an argument, it wants to pop it from the top of the stack to continue processing other arguments.

Calls that pass a block argument has it at the top of the stack, so it's the first thing YJIT needs to deal with. Now, one way to handle it is to predict where the block argument object should go and move it there. If the stack will grow like in the example on the left, moving it is fine since there is nothing at the destination.

If the stack shrinks too much, then the move would clobber another argument. On the right, 1, 2, 3 get gathered into an array for the rest parameter. That ends up where 1 is. There's a method object that needs to go on top of that, so the final location of block_arg is where 3 is. So we can't move it as the first step because we still need 3 after we're done with the block_arg. To handling this properly, YJIT needs to tuck away block_arg while processing other arguments. So it needs some temporary storage in addition to the stack, and/or it needs to move away from the typical model of always taking the top of the stack as input.



YJIT GC safety crash with proc objects as block argument

alanwu (Alan Wu) さんが約13時間前に追加.



Actually, moving up past the top of the stack isn't correct in all situations either, and YJIT has a bug there I found while writing this. The GC doesn't mark values above the top of the stack, so if we yield to the GC after the move, the block argument will be collected. Maybe YJIT needs to move away from using the stack as a scratch space during calls to handle this properly.

Argument conversion

```
foo(*arguments)  foo(**keyword_arguments)  foo(&block_argument)
```

- Triggered by caller side features. Method call before the method call!
- With [Feature #21047] (for Ruby 3.5), `nil` is special for all three and there is never calls on `nil`.
- When `to_a` returns `nil` or doesn't exist: same effect as `foo(arguments)`
 - Wrap then unwrap, like how `Array(1)` gives `[1]`, which then gets unpacked
- Conversion error for `**` and `&` instead always cause `ArgumentError`

Next I have a category that's the polar opposite in terms of complexity. When you use splat, keyword splat, and object block argument, they respectively expect a `Array`, `Hash`, and `Proc`. When the argument isn't the right type, we try to convert it to the right type by calling a conversion method. There is some differences between the 3 conversion protocols.

When you pass `nil` with these features, you end up passing nothing. `Nil`'s special status is relatively new. Block arguments treated it specially first, and it was an error for keyword splat. Ruby 3.4 made it work for keyword splat, and in the next ruby release, it should work more reliably for array splat. Right now, the pass nothing effect for single star `nil` relies on getting an empty array from calling `to_a` on `nil`, so you can redefine `to_a` on `NilClass` to break it. The next version won't have this extension point, it will always discard `nil` without calling anything.

Now, when we do call the conversion method, that can fail. The method might not exist, or if it does, it might not return the right type. Usually, we end up raising `ArgumentError`, but array splat is special. When the conversion fails, it passes the argument as if you didn't use array splat, like if you edited the code at the call site and removed the star. This effect falls out of it handling conversion failure by wrapping the object in an one-element array, and then unpacking it. Somewhat odd feature, but maybe it's just a side effect of being consistent with the capital A `Array` method on `Kernel`, which also does conversion and handles error this way.

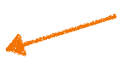
Argument conversion

& and refinements

```
class Symbol
  def to_proc = ->(x) { x.public_send(self) }
end

%w[Matsuyama EHIME].map { it.downcase } #=> ["matsuyama", "ehime"]
%w[Matsuyama EHIME].map(&:downcase)      #=> ["matsuyama", "ehime"]
```

```
class RefinedScope
  using(Module.new { refine(Kernel) { def itself = 1 } })
  [0, 0].map { it.itself } #=> [1, 1]
  [0, 0].map(&:itself)     #=> [1, 1]
end
```

 No refinements in this scope!

Now, on to block arguments. The ampersand symbol short hand is pretty well known. It's useful when you need a block that takes 1 argument and just calls a method on it. Instead of writing out the block, you can pass the name of the method using ampersand instead. I'm using downcase in the example and the two lines give the same result.

<>Great, I used to explain block argument conversion as going through the to_proc method, so for symbols, it could be defined as something like this. It returns a lambda that takes has one parameter and calls the method with the same name as the symbol. Now, my symbol to_proc implementation does work, and if you plug it in for this example, you'd get the same result, but this isn't the full story.

<>What if you have refinements? We are in a scope that redefines Kernel#itself. The usual definition just returns the receiver, so the output is identical to the input, but with this refined definition, we get "one one" instead.

Just like the downcase example, the symbol block argument shorthand behaves the same as if we wrote out the block by hand; they both use the refinement. The Symbol#to_proc reimplement actually breaks the shorthand in this case. <> The public_send in the reimplement is in a different lexical scope, so it's can't see the refinement. It might actually be impossible to reimplement Symbol#to_proc in ruby since there is no way to get lexical scopes as objects.

Argument conversion

Hypothesis: & conversion is always only calling to_proc

```
OUTSIDE = :itself.to_proc
OUTSIDE.call(0) #=> 0

class RefinedScope
  using(Module.new { refine(Kernel) { def itself = 1 } })
  OUTSIDE.call(0) #=> 1
  [0, 0].map(&OUTSIDE) #=> [1, 1]

  T0_PROC = :itself.to_proc

  def self.capture(&block) = const_set(:BLOCK_ARG, block)
  capture(&:itself)
end

[0, 0].map(&RefinedScope::T0_PROC) #=> [0, 0]
[0, 0].map(&RefinedScope::BLOCK_ARG) #=> [1, 1]
```

Here's some more evidence that symbol block arguments do more than just call to_proc. We can call to_proc on a symbol and get an identity function proc. It returns whatever you pass it. Zero in this case.

<> Now, let's have the same refined definition of itself again.

<> We can call the proc created outside refined scope inside refined scope, and the refinement does have influence, so we get 1. So far, we can still explain this as ampersand calling to_proc, just that the proc we get is refinement scope.

<> Now, let's try the opposite and call to_proc inside RefinedScope and then use the proc outside. This calls the stock itself, so we get zero zero back. This also suggests that it's not the to_proc that is influenced by the refinement, only that proc returned by it.

<> So finally, instead of calling to_proc, let's use ampersand on the symbol directly and save the proc from that. If ampersand is just converting by calling to_proc inside refined scope, we should get the same result.

<pause for dramatic effect><> But actually we get one one here. So it's doing something we can't replicate by calling to_proc. If you use refinements, the ampersand conversion process involves more than just a simple call to to_proc

Argument conversion

It's complicated

- If conversion is required, the process is quite involved
- Really, these can be broken down further into sub-categories based on the work required. But I got lazy.
- JIT bias showing here: Too complicated? Bail to interpreter.
- Bugs lurk in complexity! I found a bug while making slides, so you might get a crash when trying to run the code with Ruby 3.4.2.



Crash when proc from Symbol#to_proc called outside refinement scope

alanwu (Alan Wu) さんが約22時間前に追加.



In summary, the conversion procedure is quite complex, both when conversion succeeds and fails. Conversion happens relatively rarely though, and as we saw with splat, there are paths that require conversion but can be handled efficiently. Not all conversions require a method call. Passing a symbol as block argument usually doesn't make a method call either outside of refinement scopes, unless you redefine Symbol#to_proc.

I want to point out that just because there are ways to have a speedy implementation, though, doesn't mean the implementation is simple. The complicated parts of the semantics don't disappear, and sometimes manifest as memory usage or bugs. I found a crash writing the example program we saw.

Also, I have to admit that I'm being lazy here with the categorization by lumping all of these into the conversion category. There's quite a range of different possible behaviors that should get different categories(↗), but I'm running away from the work like when YJIT sees complex situations it can't generate good code for.

Calls with hidden data

- Intuition has it that all the callee gets are Ruby objects the caller specified, but that's not true for some call pairs.
- Calls without block to callee without a block parameter
- Call pairs with optional keyword parameters
- Call pairs with forwardable parameter, e.g. `def callee(...)`

Let's take a break and talk about something simpler. When we make a call with some arguments say, 1, 2, 3, we expect all that the callee gets is 1, 2, 3. This is not completely true, though, and sometimes there are extra data passed during calls that aren't explicitly in the arguments. The most basic example is how calls without a block usually pass some data to indicate that there is no block. That information is not a ruby object the caller explicitly passes.

Calls with hidden data

optional keyword parameters

```
def foo(a: default_a(), b: default_b(), c: default_c())
```

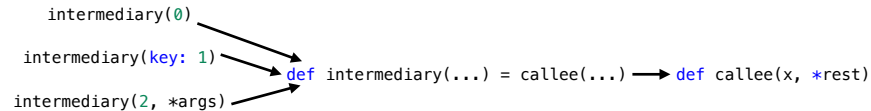
- 6 possible behaviors for computing default values
 - (a:1) (a:1, b:1) (a:1, b:1, c:1) (b:1) (b:1, c:1) (c:1)
- 2^N behaviors for N optional keyword parameters
- When $N \leq 31$, pass a bitmap. Each bit indicates whether the caller passed that keyword
- When $N > 31$, use Hash to describe the set of passed keywords
- No way to obtain either object; only the checkkeyword VM instruction uses it

When the callee has optional keyword parameters, ruby needs to compute the default value for all the keywords the caller isn't passing. For this example, there are 6 sets of calls to the default a, b, c methods to compute the default values. In general, for N optional keywords, there are 2 to the power of N ways to compute the default values. <> For each optional parameter, you can either pass it or not pass it, describable with a bit. So with enough bits, you can describe the status of all parameters. The magic number is 31 at the moment, but the point is a lot of the times the VM passes a hidden integer for optionals.

<><> If the callee has too many optional keywords, we use a Hash as a fallback, but in any case, there is no way to directly get the integer or the hash. The VM sets it up as part of the process of making the call, before running anything in the callee. The callee then checks the object for each optional keyword and run the default value computation if necessary.

Calls with hidden data

forwardable parameter



```
intermediary(0)
intermediary(key: 1)
intermediary(2, *args)
```

→ `def intermediary(...) = callee(...)` → `def callee(x, *rest)`

```
intermediary(0)    def callee(x, *rest)
intermediary(key: 1) def callee(x, *rest)
intermediary(2, *args) def callee(x, *rest)
```

Another category of call data that pass hidden data is for forwardable parameters. This started to pass hidden data in 3.4 to reduce GC allocations thanks to Aaron "tenderlove" Patterson. He coined the term forwardable and forwarding. Basically, you can define the parameter of a method that you are able to forward, so to have a forwardable parameter. <1s> You can require a parameter and not use it, but if you do use it then you are forwarding arguments to a method.

Forwarding should preserve the arguments, so it behaves as if you didn't have the intermediary triple dot method in the middle and you directly called the final method. In this example there are 3 different call pairs excluding the intermediary listed at the bottom. Different call pairs require different work to handle, so `intermediary` has each caller pass a description of the call site, along with the actual arguments. Internally this description is called `call_info` and includes info like number of arguments, and the kinds of features used.

Usually, we are done with the call info after argument passing, but in this case the call info itself is an argument. Very meta. <2s> There is no way to get the call info inside intermediary as a Ruby object though, so it's hidden data. It's only used behind the scenes during forwarding.

By the way the top diagram has the intermediary looking like a bottleneck that the 3 calls have to fit through and that's intentional. Forwarding calls are challenging to cache using the fast path mechanism mentioned earlier. Usually the call pair is good enough as a cache key but in this case you need a triplet. You have one call info that's passed for the forwardable parameter and another that describes the forwarding call.

Makes GC allocations

- Whether an operation allocates an object has always been an implementation detail since Ruby isn't designed to offer comprehensive control at this abstraction level.
- Making an allocation implicitly tasks the GC to figure out the lifetime of it.

```
def foo(*args)      def foo(*args)      def foo(*args)
  # ?               args.first          mystery_method(args)
end                 end                 end
foo(1)              foo(1)              foo(1)
```

- Alternative memory management strategies possible, but requires complex runtime analysis.

Now, I have a category for allocations and Jeremy is talking about this in the main hall, so maybe for this part we should just get up and go there for all the information we need. Anyways, I'm going to try and not overlap too much with Jeremy here. I'll start fairly abstract and work my way down to what CRuby currently does.

First of all, whether something allocates has always been up to the Ruby runtime you're using. Ruby code work with objects, but the code doesn't direct control the exact in-memory representation and the lifetime of the memory. There is no way to immediately free an ruby object, for example. All of these are up the garbage collector to figure out.

When the runtime decides to allocate an object, it's because the code needs it, yes, but also it's implicitly picking a memory management strategy that doesn't require it to know upfront when the code will be done with the new object. It doesn't know when the memory of an object can be safely freed and reused. If we know the that information, we can just deallocate the object at that time instead of asking the GC to figure it out. <2s>

In this example, I have a call pair that involves a rest parameter, and the body foo is a mystery. I want to know if is it necessary to GC allocate an array during the call. Since foo can do anything, including stashing away args in a global variable to make args live forever, we don't when, or where in code to recycle args, so we need to allocate with the GC.<>

Now, let's say we know the body of foo just returns the first item in args. We only need a representation of the array that we can call first on, and then we're done. A valid way to implement this, if we can ensure that Array#first isn't redefined, is to pass 1 directly without wrapping it in an array, and then return that.<>

Now, let's consider a third situation. We know the body of foo and it's passing args to a mystery method. The mystery method can do anything, just like foo in the first example, so we again can't tell up front where to free args again. <2s>

So in principle, sometimes it's possible to not allocate with the GC when the code needs an object to work with, but we need to analysis the ruby code to figure it out. And because you can load ruby code and make methods at runtime, this analysis needs to happen at runtime too. Also, the scope of the analysis matters. The 3 examples have visibility into different sets of methods, and that leads to different results.

It takes quite a bit of sophistication to not use the GC and get a performance win, because the analysis itself takes resources. We'll try do implement something in ZJIT.

Makes GC allocations

Quick incomplete summary



- Mostly triggered by parameter features
 - Obvious ones: rest, keyword rest, `rb_define_method` with `argc=-2`
 - Surprising cases
 - Hidden hash when there are many optional keyword parameters
 - First call to a method -- internal caches are GC managed
- Optimized cases that used to allocate but now don't
 - forwardable parameter
 - block forwarding, depending on the operation

Anyways, while on paper there are many situations we don't have to allocate with the GC, CRuby doesn't have the machinery to take advantage yet. I'm going to run through some cases that do allocate and some cases that don't thanks to optimizations. Some parameter features essentially ask for an object that contains all the arguments like rest and keyword rest, so without the kind of analysis we went over we allocate an object on those calls. By the way, native extensions can define methods that in effect have a rest parameter, so they call for an array. What's different from ruby is that the body of C methods are practically opaque for runtime analysis. So calls to these C methods will probably always require an allocation, even after ruby code can avoid allocations. <2s>

Sometimes the parameters don't explicitly require an object, but we allocate anyways. Kind of surprising. Usually, having optional keyword parameters don't cause an allocation, but if you have too many of them, we allocate a hidden hash internally. Also, for caching where method lookups land, basically all ruby calls allocate a cache object.

End

in-place hidden data
shrink conversion GC allocation

- A way to focus on a subsection of the total argument passing logic by specializing for a call pair
- An analysis to know whether a GC allocation is necessary
- 2 crashes
- My categories are arbitrary. You should make your own!
- The process of categorizing per se seems useful



We've reached the end. We saw a way to specialize parameter passing logic to a call pair, a forward looking part about avoiding allocation, and of course there's all the categories.

I'm not any kind of authority when it comes to categorizing calls, so what I showed is pretty arbitrary. In the process of conjuring these categories, though, I found 2 crashes in ruby. So this seems like a useful exercise. I think this is a good way to study the code base, and I encourage you to come up with your own categories. You might discover some exotic call animals lurking in there. Thanks for listening, and I hope you had fun at the zoo. ご清聴ありがとうございました。

Credits

- https://en.wikipedia.org/wiki/Distillation#/media/File:Simple_distillation_apparatus.svg
(public domain)
- いらすとや: <https://www.irasutoya.com/p/terms.html>